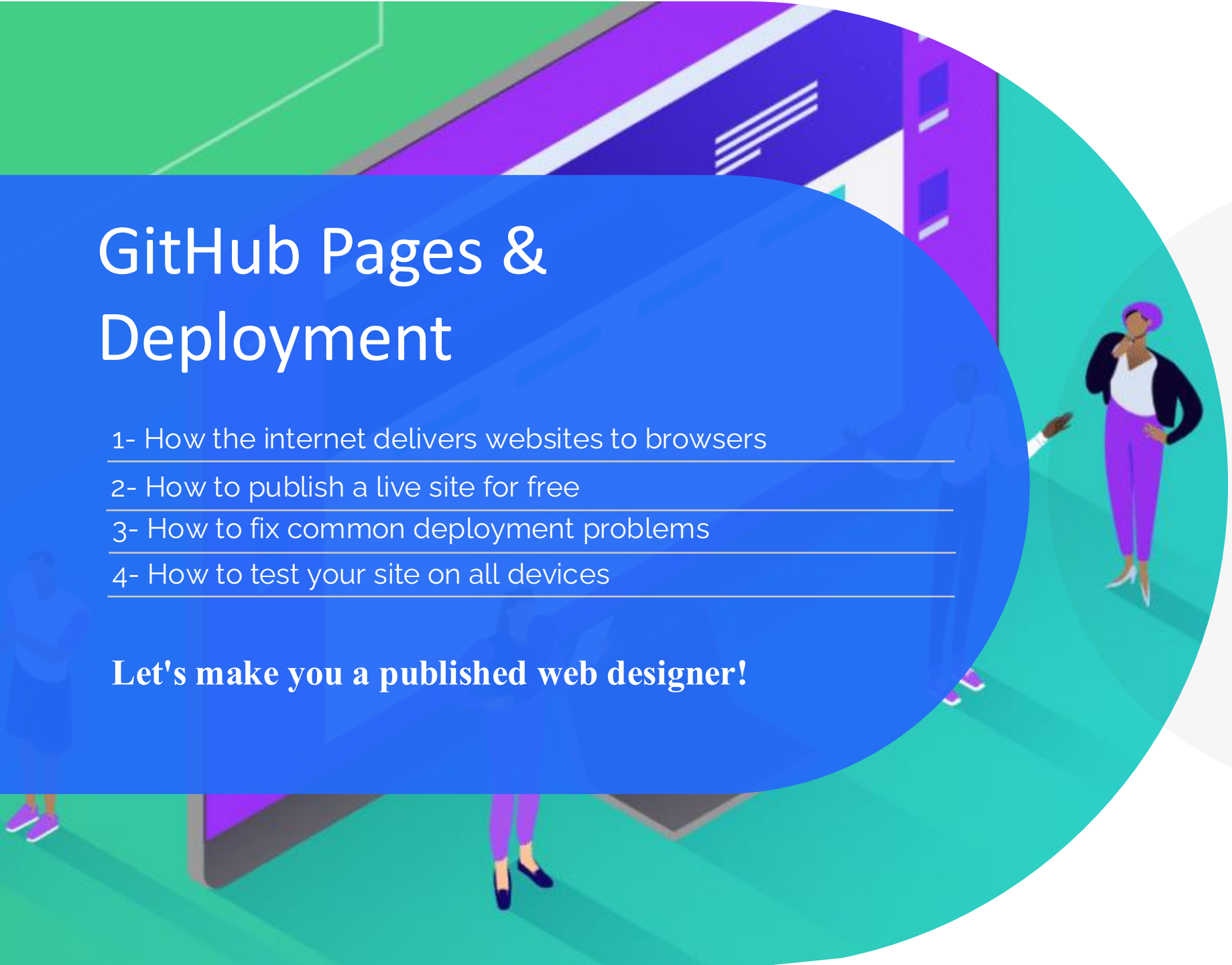


GitHub Pages & Deployment

1- How the internet delivers websites to browsers

2- How to publish a live site for free

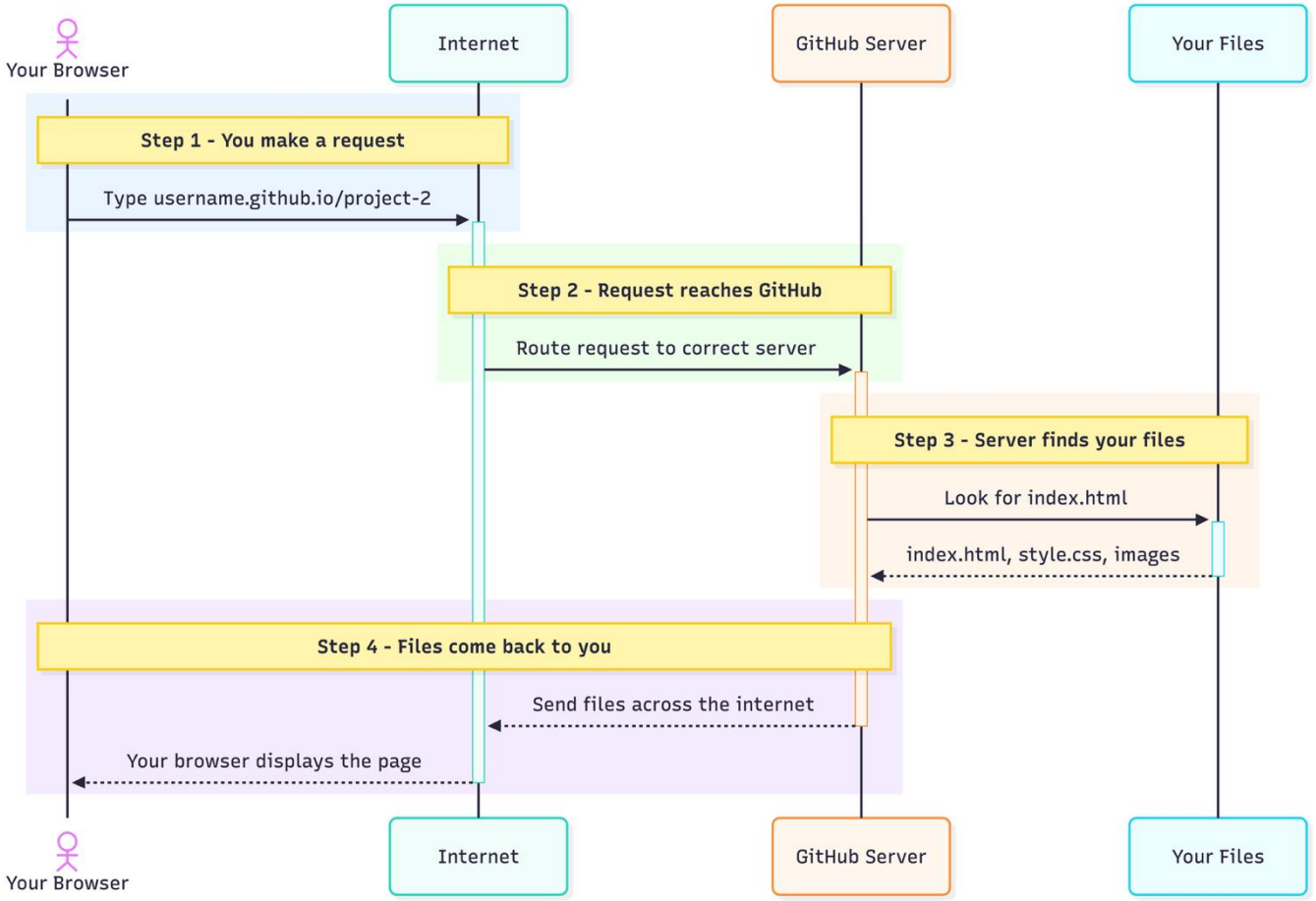
3- How to fix common deployment problems

4- How to test your site on all devices

Let's make you a published web designer!

SCROLL

How Does the Internet Work?



What Actually Happens When You Visit a Website:

- Step 1: You type a URL into your browser or click a link
- Step 2: Your browser sends a request across the internet
- Step 3: A web server receives that request
- Step 4: The server finds the right files and sends them back
- Step 5: Your browser reads those files and displays the page

A web server is just a computer that:

- Is always on and always connected to the internet
- Has files stored on it - HTML, CSS, images
- Responds to requests from browsers around the world
- Can serve thousands of visitors at the same time

What is Web Hosting?

Why Your Laptop is Not Enough:

Your laptop can technically run a website but:

- It is not always on
- It is not always connected to the internet
- It cannot handle thousands of visitors at once
- When you close VS Code, the site goes down

Web hosting solves all of that.

A web server is:

- A computer running 24 hours a day, 7 days a week
- Permanently connected to the internet with a fixed address
- Built to handle large amounts of traffic simultaneously
- Maintained by professionals so you do not have to think about it

The journey of your website:

- Week 1 to Week 10: lives only on your laptop
- Today: moves to a server and gets a real address
- From now on: available to anyone, anywhere, anytime



Hosting Options Comparison

Many Ways to Host a Website - Here is the Landscape:

Service	Cost	Best For
GitHub Pages	Free	Student projects and portfolios
Netlify	Free tier	Static sites with forms and functions
Vercel	Free tier	React, Next.js, modern frameworks
Cloudflare Pages	Free tier	Fast global delivery, great free plan
Render	Free tier	Static sites and simple backend apps
Surge.sh	Free tier	Quick command line deployments
Firebase Hosting	Free tier	Google ecosystem and web apps
Shared Hosting	$5-10 per month	WordPress and PHP sites
Kinsta	$35+ per month	Managed WordPress at scale
VPS (DigitalOcean, Linode)	$5-20 per month	Full server control
Fly.io	Free tier	Containerized apps globally
AWS (Amplify, S3, CloudFront)	Varies	Enterprise scale applications
Azure Static Web Apps	Free tier	Microsoft ecosystem projects
Google Cloud	Varies	Google ecosystem and AI workloads

Why GitHub Pages?



Why We Are Starting Here:

- Free with no credit card required, no trial period, no hidden costs
- Setup takes less than two minutes directly inside GitHub settings
- Every git push automatically updates your live site - no separate deploy step
- HTTPS is included by default -- your site is secure out of the box
- Works perfectly for HTML, CSS, and JavaScript projects - exactly what you have built
- Your code and your live site live in the same place - one tool, one workflow
- Looks professional on a resume - github.io URLs are recognized by employers
- Unlimited public repositories each get their own free deployment
- Custom domains are supported if you want to upgrade later

GitHub Pages is not just a student tool. Open source projects, developer portfolios, documentation sites, and small business sites all use GitHub Pages in production. You are learning a real tool, not a simplified version of one.

GitHub Pages Limitations

What GitHub Pages Cannot Do — And Why It Does Not Matter Yet:

Limitation	What it means	Does it affect you?
No backend languages	Cannot run PHP, Python, Ruby, Node.js on the server	No — your projects are HTML and CSS
No databases	Cannot store or retrieve user data	No — your sites are informational
No server-side logic	Cannot process forms, logins, or payments	No — not needed for class projects
1GB repository size limit	Your entire repo must be under 1GB	No — optimize your images and you will never hit this
100GB bandwidth per month	Combined traffic limit across all your Pages sites	No — you would need millions of visitors
No private repo deployment	Free GitHub Pages requires a public repository	Minor — your code will be publicly visible

GitHub Pages is purpose-built for static sites — HTML, CSS, and JavaScript with no server needed. Every project you have built in this class is a static site. The limitations only become relevant when you start building web applications with user accounts, databases, and server-side processing.

Static vs Dynamic Sites

Static Site:

- Files are pre-built and stored on the server
- Every visitor gets the exact same files
- No processing happens on the server
- Extremely fast and cheap to host
- Perfect for portfolios, news sites, documentation
- Examples: your class projects, GitHub Pages sites, marketing pages

Dynamic Site:

- Pages are generated on demand for each visitor
- Content changes based on who is logged in
- Requires a server running code at all times
- More complex and expensive to host
- Necessary for user accounts, shopping carts, social feeds
- Examples: Instagram, Gmail, Amazon, Netflix

Where it gets interesting in 2026:

The line between static and dynamic has blurred. Tools like Next.js, Astro, and SvelteKit can pre-build pages that feel dynamic — pulling in content from databases at build time rather than at request time. Many modern sites that look dynamic are actually serving static files with JavaScript handling the interactive parts. This approach is called a JAMstack architecture and it is increasingly common in professional web development.

Creating a GitHub Pages



Getting Started with GitHub Pages

Three Things You Need Before We Deploy:

1. Your repository is on GitHub

- Log in to github.com and confirm your project-2 repo is visible
- Click into it and make sure your files are there; not an empty repo

2. index.html is in the root folder

- Click into your repo on GitHub and look at the file list
- index.html must appear at the top level; not inside a subfolder
- If it is inside a folder called my-site or project or anything else, that is a problem we need to fix before deploying

3. Everything is committed and pushed

- On GitHub, your latest changes should be visible
- If you made changes last night that are not showing, you need to push before deploying
- Run git status in your terminal — it should say nothing to commit, working tree clean

Critical File Structure

Your File Structure Will Make or Break Your Deployment:

Correct: index.html at the root level:

```
project-name/
├── index.html
├── about.html
├── article.html
├── contact.html
├── README.md
├── .gitignore
├── assets/
│   ├── css/
│   │   ├── style.css
│   │   └── reset.css
│   ├── images/
│   │   ├── hero.webp
│   │   └── profile.webp
└── docs/
    └── case-study.pdf
```

Three rules that never change:

- All HTML files sit at the root level — GitHub Pages requires this
- Everything that is not HTML lives inside assets with its own subfolder
- PDFs and downloadable files go inside docs — never loose in the root

How to check your own structure right now:

- Open your repo on github.com
- index.html must be visible at the top level

Why this matters:

GitHub Pages looks for index.html at the root of your repository. If it finds a folder instead, it has nothing to display and returns a 404 error. The files inside that subfolder are invisible to GitHub Pages entirely.

Step 1 - Go to Repository Settings

Open your project repository on GitHub and click **Settings.**
Inside Settings, follow these steps in order:

Step 1 — Confirm your repo is public:
- Scroll down to the Danger Zone section
- Look for the visibility setting
- If it says Private, click "Change visibility" and switch to Public
- GitHub Pages does not work with private repos on a free account

Step 2 — Find GitHub Pages:
- Go back to the top of the left sidebar
- Click **Pages** under Code and automation

Step 3 — Configure your source:
- Under Source, select "Deploy from a branch"
- Branch: choose main
- Folder: choose / (root)
- Click Save

Step 4 — Wait for deployment:
- GitHub takes 1 to 2 minutes to build your site
- Refresh the Pages settings screen after a minute
- You will see your live URL appear at the top

All four steps happen inside Settings without leaving the page.

Step 2 - Your Site is Live!

Refresh the Pages Settings Screen After 1 to 2 Minutes:
You will see a green banner at the top that says:
"Your site is live at https://username.github.io/project/"

What you should see:
- Your actual Project website
- Loading from a real URL that anyone can visit
- Secure HTTPS connection - the padlock icon is in the address bar

What to do immediately:
- Copy your URL and save it somewhere safe
- Open it in a new incognito window to see it exactly as a visitor would
- Text it to someone - your site is on the internet

If you see a 404 error instead:
- Wait another minute and refresh - deployment sometimes takes longer
- Check that index.html is at the root level of your repo
- Check that your branch is set to main not master
- Check the Actions tab on your repo for deployment error details

If your URL appears but the page looks broken:
- Your file paths are likely absolute instead of relative
- We will cover exactly how to fix this in the next few slides

Understanding Your URL

Your URL is Not Random - Every Part Means Something:

https://username.github.io/project-2/
| | | | |
secure your GitHub username GitHub hosting your repo

Breaking it down:

- **https:** your site is secure by default, no setup required
- **username:** your GitHub username, which is why choosing it carefully mattered
- **github.io:** GitHub's hosting domain, tells the world this is a GitHub Pages site
- **project-2:** the exact name of your repository on GitHub

Two types of GitHub Pages URLs:

Project site - what you have now:

username.github.io/project-2
username.github.io/project-3
username.github.io/anything

One URL per repository, unlimited projects.

Personal site - one per account:

username.github.io

Create a repo named exactly your username and it becomes your root URL. No subfolder. This is what your portfolio will use.

Updating Your Live Site

Your Site Updates Automatically Every Time You Push:

Make a change, save, then run these four commands:

```
git add .
git commit -m "Update homepage headline"
git push
```

Then wait 1 to 2 minutes and refresh your live URL.

What happens behind the scenes when you push:

- GitHub receives your new commits
- GitHub Actions automatically triggers a build
- Your updated files replace the old ones on the server
- Your live URL reflects the changes within minutes

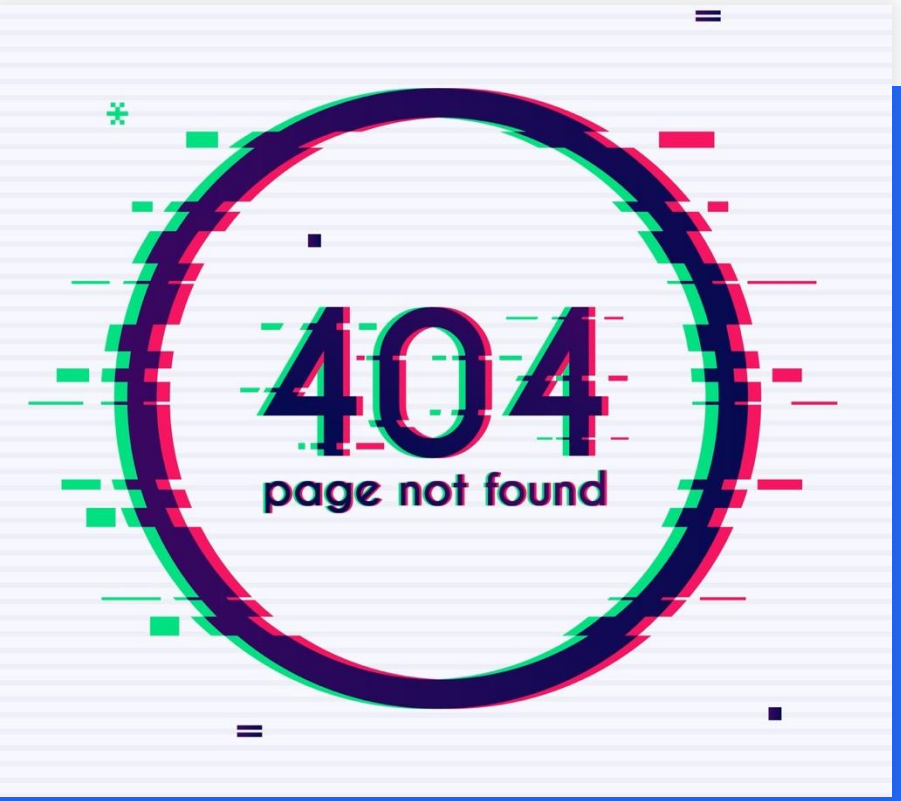
Good habits for updating a live site:

- Always test changes locally in VS Code before pushing
- Write a clear commit message describing exactly what you changed
- Check the Actions tab if your changes do not appear after 3 minutes
- Hard refresh your browser with Cmd+Shift+R on Mac or Ctrl+Shift+R on Windows - browser cache is usually the reason changes seem missing

Common Deployment Errors



Problem 1: 404 Error



A 404 error means GitHub Pages could not find your index.html file.

Check 1: File structure

•Open your repo on github.com

- Can you see index.html at the root level?
- If it is inside a subfolder, move it up and push again

Check 2: Branch settings

- Go to Settings, then Pages
- Confirm the branch is set to main not master
- Confirm the folder is set to / (root) not /docs

Check 3: Filename spelling

- The file must be named exactly index.html
- Not Index.html, not index.HTML, not homepage.html
- GitHub servers are case sensitive, your laptop is probably not

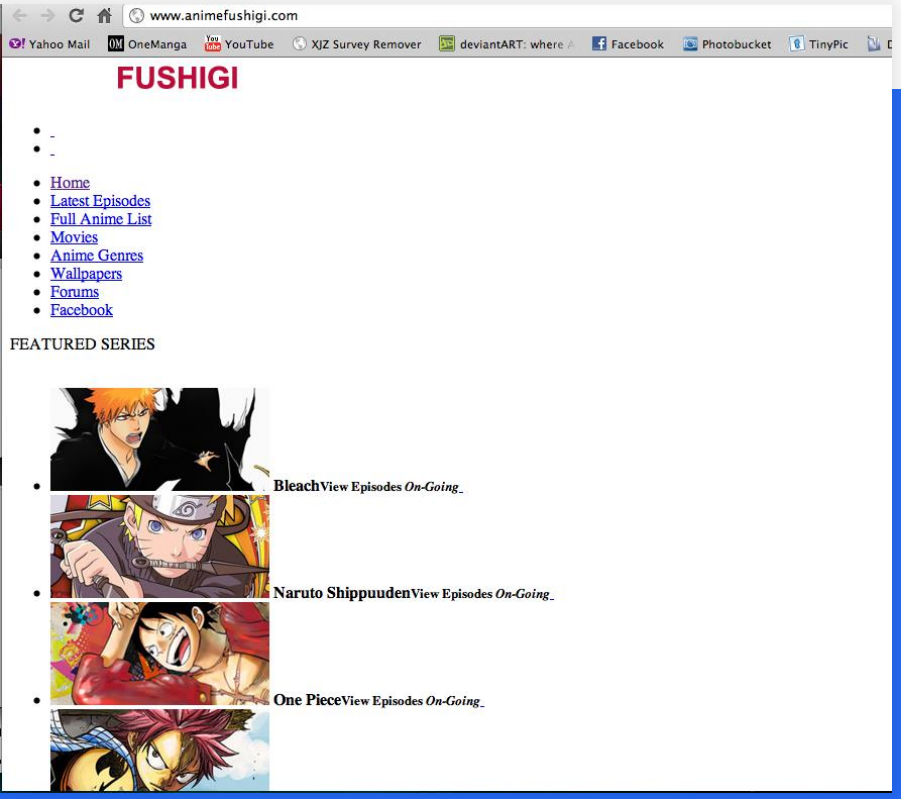
Check 4: Deployment timing

- GitHub Pages takes 1 to 2 minutes to deploy
- Check the Actions tab, is there a green checkmark or a red X?
- If the Actions tab shows a red X, click it for specific error details

Check 5: Repository visibility

- Go to Settings and confirm the repo is Public not Private
- Private repos do not deploy on free GitHub accounts

Problem 2: Styles Not Loading



Your Site Loads But Looks Completely Unstyled:

This is almost always a file path problem. Your CSS file exists but your HTML cannot find it.

The rule: always use relative paths, never absolute paths.

Absolute paths: broken on GitHub Pages

```
<!-- Points to your laptop, not the server -->
<link href="C:/Users/Me/Desktop/project-2/assets/css/style.css">

<!-- Points to the root of the domain, skips your repo name -->
<link href="/assets/css/style.css">
```

Relative paths: work everywhere

```
<!-- Goes into assets, then css, then style.css -->
<link href="assets/css/style.css">

<!-- The ./ means start from the current file location -->
<link href="./assets/css/style.css">
```

Problem 3: Images Not Loading

Your Pages Load But Images Show Broken Icons:

Images not loading on GitHub Pages almost always comes down to one of four causes. Check them in this order:

Check 1: File path

```
<!-- Wrong: absolute path -->
<img src="C:/Users/Me/Desktop/project-2/assets/images/hero.jpg">

<!-- Wrong: leading slash -->
<img src="/assets/images/hero.jpg">

<!-- Correct: relative path -->
<img src="assets/images/hero.webp">
```

Check 2: Case sensitivity GitHub servers are case sensitive. Your laptop is probably not.

```
<!-- Your HTML says: -->
<img src="assets/images/Hero.jpg">

<!-- Your actual file is named: -->
hero.jpg

<!-- GitHub cannot find it. Your laptop can. Rename to match exactly. -->
```

Problem 3: Images Not Loading cont.

Your Pages Load But Images Show Broken Icons:

Images not loading on GitHub Pages almost always comes down to one of four causes. Check them in this order:

Check 3: File was never pushed

- Go to your repo on github.com
- Click into assets, then images
- If your image file is not listed there, it was never pushed
- Run git add ., git commit, and git push then check again

Check 4: File format

- Are you using modern formats like WebP?
- Older browsers have limited WebP support but all modern browsers handle it fine
- If an image works locally but not on a specific device, check the format

A good habit going forward:

- Name all image files in lowercase with no spaces
- Use hyphens between words: hero-image.webp not Hero Image.webp
- Commit and push images the same time you write the HTML that references them

Problem 4: Changes Not Showing

You Pushed Your Changes But the Live Site Looks the Same:

Before assuming something is broken, work through these checks in order:

Check 1: Did you actually push?

```
git status
```

If it says "nothing to commit, working tree clean" and your changes are not on GitHub, you forgot to push. Run:

```
git push
```

Check 2: Hard refresh your browser Regular refresh shows a cached version. Hard refresh forces a fresh download.

- Mac: Cmd + Shift + R
- Windows: Ctrl + Shift + R

Check 3: Open in incognito or private mode

- Mac: Cmd + Shift + N in Chrome
- Windows: Ctrl + Shift + N in Chrome Incognito has no cache at all and shows exactly what a first time visitor sees.

Check 4: Check the Actions tab

- Open your repo on github.com
- Click the Actions tab
- Look for your most recent deployment
- Green checkmark means it finished successfully
- Spinning yellow circle means it is still building, wait another minute
- Red X means something went wrong, click it for details

Testing & Deployment Options



Mobile Testing is Critical

Your Site is Live — Now Make Sure it Works Everywhere:

A site that looks great on your laptop but breaks on a phone is not a finished site.

Method 1: Your actual phone (most important)

- Open your GitHub Pages URL on your phone
- This is the only test that shows exactly what real visitors experience
- Check every page, every link, every image
- Try both portrait and landscape orientation

Method 2: Chrome DevTools device emulation

- Open your live site in Chrome
- Right click anywhere and select Inspect
- Click the device icon in the top left of DevTools
- Choose a device from the dropdown: iPhone 14, Samsung Galaxy, iPad
- Resize and test at multiple screen sizes

Method 3: Resize your browser window

- Drag your browser window narrower
- Watch for layout breaking points
- Note where content overlaps, overflows, or disappears

What to look for on every device:

- Navigation is usable and not overflowing
- Text is readable without zooming
- Images scale correctly and do not overflow their containers
- Buttons and links are large enough to tap with a finger
- No horizontal scrolling on mobile

Performance Check with Lighthouse

Your Site is Live and Mobile Tested -- Now Measure Its Quality:

Lighthouse is a free tool built into Chrome that audits your site and gives it a score out of 100 in four categories.

How to run it on your live site:

- Open your GitHub Pages URL in Chrome
- Right click anywhere and select Inspect
- Click the Lighthouse tab at the top of DevTools
- Under Categories, check all four boxes
- Under Device, select Mobile first, then run again on Desktop
- Click Analyze page load

Reading your scores:

Score	Color	What it means
90 to 100	Green	Excellent
50 to 89	Orange	Needs improvement
0 to 49	Red	Poor, fix before presenting

Custom Domains (Optional)

Want a Professional URL Instead of a GitHub Pages Address?

This is completely optional for class but strongly recommended for your portfolio. How it works in three steps:

Step 1: Buy a domain

- Choose a registrar and purchase your domain
- yourname.com is ideal for a portfolio
- Cost is typically $10 to $15 per year

Step 2: Configure DNS

- In your domain registrar settings, add four A records pointing to GitHub Pages servers:
  185.199.108.153
  185.199.109.153
  185.199.110.153
  185.199.111.153
- Add a CNAME record pointing www to username.github.io

Step 3: Tell GitHub your domain

- Go to Settings, then Pages in your repo
- Enter your custom domain in the Custom domain field
- Check Enforce HTTPS
- Create a file called CNAME in your repo root containing just your domain name:
  yourname.com

DNS changes take 24 to 48 hours to fully propagate worldwide.

Where to Buy Domains

Choosing Where to Buy Your Domain:

Registrar	First Year	Renewal	WHOIS Privacy	Verdict
Cloudflare	$8 to $10	Same price	Free	Best price, no markup ever
Porkbun	$9 to $11	Same price	Free	Best for beginners, no upsells
Namecheap	$10 to $14	Same price	Free	Reliable, well established
GoDaddy	$1 to $2	$20 to $25	Paid extra	Avoid, renewal price trap
Squarespace Domains	$20+	$20+	Free	Overpriced, was Google Domains

One rule: avoid hyphens. yourname.com is better than your-name.com in every way.

Other Deployment Options

Beyond GitHub Pages: The Deployment Landscape in 2026

Netlify Best for: static sites with added functionality

- Drag and drop deployment, no terminal required
- Automatic deployments from GitHub
- Built in form handling without a backend
- Serverless functions for simple dynamic features
- Free tier is genuinely generous
- netlify.com

Vercel Best for: modern JavaScript frameworks

- Built by the team behind Next.js
- Extremely fast global delivery network
- Instant preview URLs for every git push
- Best choice when you start learning React or Next.js
- Free tier covers most personal projects
- vercel.com

Cloudflare Pages Best for: speed and global performance

- Fastest delivery network available in 2026
- Unlimited bandwidth on free tier
- Excellent built in analytics
- Integrates with Cloudflare Workers for serverless functions
- pages.cloudflare.com

Render Best for: when you need a simple backend

- Hosts both static sites and simple server applications
- Free tier available for static sites
- Good bridge between GitHub Pages and full server hosting
- render.com

FTP Deployment

The Traditional Way to Deploy: FTP

FTP stands for File Transfer Protocol. It is how websites were deployed before Git and automated deployment existed, and it is still widely used today in certain contexts.

How it works:

- You build your site locally in VS Code
- You open an FTP client and connect to your web server
- You manually drag and drop your files to the server
- Visitors can now access your site

The tools:

- FileZilla: free, cross platform, most widely used FTP client
- Cyberduck: free, clean interface, popular on Mac
- VS Code has FTP extensions that let you deploy without leaving the editor

FTP compared to GitHub Pages:

	FTP	GitHub Pages
Deployment method	Manual drag and drop	Automatic on git push
Version control	None built in	Full Git history
Speed	As fast as your connection	1 to 2 minutes
Cost	Requires paid hosting	Free
Still used in industry	Yes	Yes


Deployment not working?
Lighthouse scores too low?
404 error you cannot fix?
Custom domain questions?
Mobile layout breaking?
Anything unclear?
HAVE ANY
QUESTIONS?